The functional requirements will be designed as use cases to clearly describe the interactions between system actors and the Ticket Reservation System and to show how the system behaves in response to user actions.

ID:	UC-01
Title:	Customer Registration
Primary Actor(s):	Customer
Description:	This use case describes how a customer registers as a new user in the Ticket Reservation System.
Preconditions:	● The customer has access to a device with internet. ● The customer does not have an existing account. ● The reservation system is online and running.
Postconditions:	● The customer's account is created in the database. ● The customer can log into the system.
Inputs:	Username, password, email, or phone number.
Outputs:	Confirmation message for successful registration.
Main Success Scenario:	1. The customer opens the application and selects register. 2. The system prompts the customer to enter register information. 3. The customer enters all required details. 4. The system validates the input. 5. The system creates a new account in the database. 6. A confirmation message is displayed to show successful registration.

ID:	UC-02
Title:	User Login
Primary Actor(s):	Customer or Administrator
Description:	This use case describes how Customers and Administrators log into the system using their credentials.
Preconditions:	● User has access to a device with internet. ● The system is online and running. ● The user must already have an account. ● The administrator has preconfigured details.
Postconditions:	The user is granted access to their respective dashboard.
Inputs:	Username, password
Outputs:	Confirmation message for successful login.
Main Success Scenario:	1. User selects Login. 2. System prompts for credentials. 3. User enters username and password. 4. System validates credentials. 5. System redirects the user to the dashboard.

ID:	UC-03
Title:	Browse Events
Primary Actor(s):	Customer
Description:	This use case describes how a Customer views events in the system.
Preconditions:	1. The ticket reservation system is online and operational.

	2. At least one event exists in the system database.
Postconditions:	1. Available events are displayed to the customer. 2. Each event has its own unique details (name, date, location, price, etc.)
Inputs:	None
Outputs:	Event list with details
Main Success Scenario:	1. Customer navigates to the Events page. 2. System retrieves active events from the database. 3. The system sorts events by upcoming date. 4. The system displays the event list to the customer. 5. The customer may select an event for more details.

ID:	UC-04
Title:	Search and Filter Events
Primary Actor(s):	Customer
Description:	This use case describes how a Customer searches for and filters events by price, date, location, etc.
Preconditions:	● Events exist in the system database. ● Customer is viewing the event list.
Postconditions:	● Only matching events are displayed.
Inputs:	Search, category, location, dates, etc.
Outputs:	Filtered list of events
Main Success Scenario:	1. Customer enters search keywords or selects filters.

	2. System validates filter values. 3. System queries the database using the search parameters. 4. The system removes the events that don't match the criteria. 5. System displays filtered event list. 6. If no events are found, the system displays the "No events found" message.

ID:	UC-05
Title:	View Event Details
Primary Actor(s):	Customer
Description:	This use case describes how a Customer views detailed information about a selected event.
Preconditions:	● Selected event exists ● Customer is logged in
Postconditions:	● Detailed event information is displayed
Inputs:	EventID
Outputs:	Event description (available seats, price, etc)
Main Success Scenario:	1. The customer selects an event from the list. 2. System retrieves event information. 3. System retrieves seat availability. 4. System displays full event information. 5. The customer can proceed to reserve the event.

ID:	UC-06
Title:	Reserve Ticket
Primary Actor(s):	Customer
Description:	This use case describes how a Customer reserves a ticket for an event.
Preconditions:	● Customer is logged in ● Event exists ● Seats are available
Postconditions:	● A reservation is created and associated with the customer. ● Selected seats are marked as reserved. ● The reservation expiry timer is created. ● Confirmation notification
Inputs:	CustomerID, EventID, SeatNumber, Payment
Outputs:	ReservationID, Updated seats
Main Success Scenario:	1. The customer selects the event and chooses seats. 2. System checks seat availability. 3. System locks the seat temporarily since the application must allow for concurrent users. 4. Customer confirms booking. 5. The system creates a reservation with details. 6. System updates seat status to reserved. 7. The system generates a reservation confirmation. 8. Customer receives confirmation via email / SMS

ID:	UC-07
Title:	Cancel Reservation
Primary Actor(s):	Customer

Description:	This use case describes how a Customer cancels an existing reservation.
Preconditions:	● Customer is logged in ● Reservation exists and is valid
Postconditions:	● Reservation status changed to Cancelled. ● Seats are released and marked available. ● Refund process initiated (if applicable). ● Notification sent to customer.
Inputs:	ReservationID
Outputs:	Updated ReservationStatus, Updated seats availability
Main Success Scenario:	1. Customer opens booking history 2. Customer selects a reservation 3. Customer selects “Cancel reservation” 4. System verifies cancellation policy 5. System updated reservation status to Cancelled 6. System releases reserved seats 7. System processes refunds 8. System sends cancellation confirmation notification

ID:	UC-08
Title:	Add Event
Primary Actor(s):	Administrator
Description:	This use case describes how an Administrator creates a new event in the system.
Preconditions:	● Administrator is logged in ● System is online
Postconditions:	● New event is stored in database ● Event visible to customers

Inputs:	EventName, Event Description, date, venue, seatCapacity, pricing information, category
Outputs:	EventID, Confirmation message
Main Success Scenario:	1. Administrator access admin dashboard 2. Administrator enters event details 3. System validates entry 4. System checks date conflicts 5. System creates new event record 6. Event status set to Active 7. Confirmation message displayed

ID:	UC-09
Title:	Edit Event
Primary Actor(s):	Administrator
Description:	This use case describes how an Administrator modifies an event.
Preconditions:	● Administrator is logged in ● Event exists
Postconditions:	● Event information updated ● Update reflected in dashboard
Inputs:	EventID, updated event details
Outputs:	Updated event record
Main Success Scenario:	1. Admin selects existing event 2. System modifies event details 3. System validates changes 4. System updates event record in database 5. Confirmation message is displayed

ID:	UC-10
Title:	Cancel Event
Primary Actor(s):	Admin
Description:	This use case describes how an Administrator cancels an existing event.
Preconditions:	● Admin is logged in ● Event exists and is active
Postconditions:	● Event status set to cancelled ● All reservations related to event are cancelled ● Customers are notified and refunded(if applicable)
Inputs:	EventID
Outputs:	Updated EventStatus, refunds, customers notified
Main Success Scenario:	1. Admin selects an event 2. Admin selects “Cancel Event” 3. System validates cancellation 4. System updates event status to Cancelled 5. System updates associated reservations 6. System notifies affected customers 7. Event removed from active listings

Non-Functional Requirements

Non-functional requirements are criteria used to judge the operation of a system. Listed below are the following non-functional requirements for our system: performance, availability and usability.

1. **Performance requirements:** The system should support concurrent users without performance degradation

ID	Category	Requirement Description	Measurement Metric	Target Value	Justification
NFR-P1	Performance	The system shall support simultaneous users during peak usage.	Number of concurrent active sessions	≥ 100 concurrent users	Ticket sales for popular events may generate high traffic; The system must remain responsive.
NFR-P2	Performance	The system shall process reservation requests efficiently.	Average response time for booking transaction	≤ 2 seconds	Slow booking increases user frustration and abandonment rate.
NFR-P3	Performance	Event search and filtering shall execute efficiently.	Average search query response time	≤ 1.5 seconds	Fast browsing improves user experience and usability.

2. **Availability:** The system should be cloud based that ensures high availability

ID	Category	Requirement Description	Measurement Metric	Target Value	Justification
NFR-A1	Availability	The system shall maintain continuous service availability.	System uptime percentage	≥ 99% uptime	Users must be able to book tickets at any time.

3. **Usability:** The UI should be simple and user-friendly

ID	Requirement	Metric	Target	Justification
NFR-U1	Users shall complete ticket booking with minimal steps.	Number of steps	≤ 5 steps	Reduces cognitive load and improves efficiency.
NFR-U2	The system shall provide clear confirmation feedback.	Confirmation visibility	Immediate on-screen confirmation	Users require assurance that booking was successful.
NFR-U3	The system shall minimize user input errors.	Form validation errors	Client-side validation before submission	Prevents incomplete or incorrect bookings.